

LIVRE BLANC · GUIDE PRATIQUE · 2025

Le Guide du Code Assisté par IA

*Bâtir et Déployer des Applications
sans Savoir Coder*

De la conception à la mise en ligne, en passant par le mobile — tout ce qu'un non-développeur doit savoir pour créer de vraies applications grâce aux grands modèles de langage. Un guide actionnable, dense et sans jargon inutile.

Claude · GPT-4 · Gemini

HTML / CSS / JS

Netlify · Vercel

PWA · APK

Table des Matières

CHAPITRE 01

Introduction : La Révolution du Code par IA

Pourquoi Claude et les LLM récents changent tout pour les non-développeurs

3

CHAPITRE 02

Le Cahier des Charges Technique

Structurer ses demandes · La méthode SPEC · Le prompt système infallible

6

CHAPITRE 03

Le Workflow de Prototypage

De la génération de code à la prévisualisation interactive

10

CHAPITRE 04

L'Hébergement & la Mise en Ligne

Packager son code · Netlify · Vercel · Domaine · Sécurité

15

CHAPITRE 05

L'Extension Mobile

PWA · APK Android · Google Play Store · iOS

18

CHAPITRE 06

Cheat Sheet Finale

Règles d'or · Erreurs fatales · Template · Lexique · Ressources

22

CHAPITRE 01 — INTRODUCTION

La Révolution du Code par IA

En moins de trois ans, les grands modèles de langage ont transformé le développement logiciel. Ce chapitre explique pourquoi cette transformation est irréversible — et comment vous pouvez en bénéficier dès aujourd'hui, sans une seule ligne de code préalable.

Pourquoi Claude et les LLM changent tout

Pendant des décennies, créer une application web ou mobile nécessitait des mois de formation, la maîtrise de plusieurs langages de programmation et une compréhension profonde des architectures système. Ce modèle a volé en éclats avec l'émergence des LLM (Large Language Models) de nouvelle génération.

Des modèles comme Claude 3.5 Sonnet, GPT-4o ou Gemini 1.5 Pro ne se contentent plus de compléter du texte : ils **comprennent le contexte fonctionnel** d'une application, proposent une architecture cohérente, génèrent du code commenté, détectent leurs propres erreurs et itèrent en fonction de vos retours en langage naturel. La barrière entre "avoir une idée" et "avoir une application" s'est réduite à quelques échanges bien structurés.

Claude en particulier s'est imposé comme le modèle de référence pour le codage assisté, grâce à plusieurs caractéristiques distinctives : une fenêtre de contexte étendue permettant de travailler sur des bases de code entières, une capacité supérieure à expliquer le code qu'il génère, et une fiabilité accrue dans le respect des contraintes techniques exprimées en langage naturel.

Le mythe du "il faut savoir coder"

Ce mythe mérite d'être décomposé. Techniquement, pour *maintenir* une base de code professionnelle à long terme, une compétence technique reste un avantage. Mais pour **concevoir, prototyper, déployer et itérer** sur une application fonctionnelle destinée à des utilisateurs réels ? L'IA a rendu cette exigence obsolète dans la majorité des cas d'usage.

Ce que vous devez maîtriser à la place est bien plus accessible : la pensée systémique (décomposer un problème en modules), la communication précise (décrire ce que vous voulez avec clarté) et la validation fonctionnelle (savoir si une application fait ce qu'elle est censée faire). Ces compétences sont universelles.

💡 À QUI S'ADRESSE CE GUIDE ?

Ce guide est conçu pour :

- **Entrepreneurs et solopreneurs** qui veulent valider un concept sans budget dev
- **Consultants et freelances** qui souhaitent automatiser leurs livrables ou créer des outils clients
- **Créatifs et designers** qui veulent donner vie à leurs maquettes sans dépendre d'un développeur
- **Étudiants et apprenants** qui découvrent l'IA et veulent créer des projets concrets
- **Product managers et chefs de projet** qui veulent prototyper avant de mobiliser une équipe technique

Ce que permet réellement l'IA en 2025

Voici ce que les LLM actuels permettent concrètement dans le domaine du développement applicatif :

- **Génération complète d'applications mono-fichier** (HTML/CSS/JS) en une seule conversation
- **Débogage assisté** : coller un message d'erreur suffit pour obtenir un diagnostic et une correction
- **Refactorisation guidée** : "transforme cette app en version responsive pour mobile" — l'IA exécute
- **Architecture système** : choisir entre une base de données locale (localStorage), une API REST ou Firebase
- **Intégration d'APIs tierces** : connecter Stripe, SendGrid, OpenAI ou n'importe quelle API documentée
- **Génération de tests** et de documentation technique automatique
- **Traduction inter-technologies** : convertir du Python en JavaScript, du React en Vue, etc.

⚠️ LIMITES À CONNAÎTRE

L'IA n'est pas infallible. Elle peut générer du code fonctionnel mais non sécurisé, produire des erreurs dans les applications très complexes, ou adopter de mauvaises pratiques si le prompt est vague. Ce guide vous apprend précisément à éviter ces pièges.

Les 3 paradigmes du code assisté par IA

01

Vibe Coding

Approche conversationnelle et intuitive où l'on décrit ses envies en langage naturel, sans structure formelle. Idéal pour les prototypes rapides et l'exploration. Risque : manque de cohérence sur les projets longs.

02

Prompt Engineering pour le Code

Approche structurée qui consiste à rédiger des prompts précis, avec contexte, contraintes et exemples. Produit des résultats bien plus stables et maintenables. C'est le cœur de ce guide.

03

No-Code Augmenté

Utilisation d'outils no-code (Webflow, Bubble, Glide) combinée à l'IA pour personnaliser les parties que ces plateformes ne permettent pas de modifier nativement — le meilleur des deux mondes.

Dans ce guide, nous nous concentrons principalement sur le **Prompt Engineering pour le Code**, qui offre le meilleur équilibre entre puissance, contrôle et reproductibilité. Vous apprendrez à construire des prompts qui produisent des applications fiables, extensibles et déployables.



LA PROMESSE DE CE GUIDE

À l'issue de ce guide, vous serez capable de : concevoir le cahier des charges d'une application, générer son code complet par IA, le tester localement, le déployer en ligne avec une URL publique, et le transformer en application mobile installable — le tout sans écrire une seule ligne de code manuellement.

CHAPITRE 02

Le Cahier des Charges Technique

La qualité du code généré par l'IA est directement proportionnelle à la qualité de votre brief. Ce chapitre vous donne une méthode systématique pour structurer vos demandes et obtenir du code production-ready dès le premier échange.

Pourquoi un CDC est indispensable

Une erreur classique du débutant : demander directement "crée-moi une app de gestion de projet" et espérer un résultat parfait. Ce que l'IA reçoit est une instruction vague sur laquelle elle va extrapoler — parfois brillamment, souvent de façon décalée par rapport à vos attentes.

Un Cahier des Charges (CDC) bien rédigé transforme radicalement la qualité du résultat. Il joue plusieurs rôles simultanément : **guider l'IA** dans ses choix architecturaux, **contraindre ses libertés créatives** dans les domaines où vous avez des préférences précises, et **fournir un référentiel de validation** pour vérifier que le code produit correspond à vos besoins.

La méthode SPEC

La méthode SPEC est un cadre en quatre dimensions pour structurer tout projet de développement assisté par IA :

S**Scope — Périmètre**

Qu'est-ce que l'application doit faire, exactement ? Listez les fonctionnalités obligatoires (MVP) et distinguez-les des fonctionnalités secondaires. Soyez précis sur ce qui est IN et ce qui est OUT.

P**Profil — Utilisateurs**

Qui utilisera l'application ? Un seul utilisateur sur desktop ? Plusieurs utilisateurs sur mobile ? Des personnes peu à l'aise avec la tech ? Le profil dicte l'UX, l'interface et les contraintes d'accessibilité.

E**Environnement — Stack**

Quelles technologies utiliser ? HTML/CSS/JS pur (recommandé pour débiter) ? React ? Une base de données ? Une API externe ? Précisez aussi l'environnement d'exécution : navigateur, mobile, serveur.

C**Contraintes**

Ce qui ne doit PAS se passer : pas de dépendances externes, tout en un seul fichier, design dans telle palette de couleurs, compatibilité avec tel navigateur. Les contraintes éliminent les mauvaises surprises.

Les 5 questions à répondre avant tout prompt

- **1. Quel est l'objectif principal de l'utilisateur ?** Que cherche-t-il à accomplir en 30 secondes ?
- **2. Quelle est la donnée centrale ?** Qu'est-ce que l'app crée, lit, met à jour ou supprime ?
- **3. Où vit la donnée ?** localStorage, fichier JSON, base de données, API distante ?
- **4. Quel est le look & feel attendu ?** Palette de couleurs, style (minimaliste, coloré, corporate) ?
- **5. Quelles sont les contraintes absolues ?** Mobile-first, accessibilité, un seul fichier HTML, etc. ?

💡 LE PRINCIPE DES MICRO-TÂCHES

Ne demandez jamais à l'IA de construire une application entière en un seul prompt si elle comporte plus de 5 fonctionnalités distinctes. Décomposez-la en modules indépendants et construisez-les un par un. Cette approche réduit drastiquement les erreurs et facilite le débogage. Par exemple : d'abord la structure HTML, puis le style CSS, puis la logique JS, puis la sauvegarde des données, puis les formulaires.

Le prompt système de développement infallible

Voici le template de prompt système à injecter en début de conversation pour cadrer l'IA comme un développeur senior :

📄 TEMPLATE — PROMPT SYSTÈME DÉVELOPPEUR

RÔLE

Tu es un développeur full-stack senior spécialisé dans la création d'applications web légères, modernes et accessibles aux non-développeurs. Tu codes proprement, tu commentes tes blocs importants et tu expliques chaque décision architecturale en une phrase.

RÈGLES ABSOLUES

1. Tout le code doit tenir dans UN SEUL fichier HTML autonome.
2. Utilise uniquement HTML5, CSS3 et JavaScript vanilla (pas de framework).
3. Les données sont stockées en localStorage sauf indication contraire.
4. Le design doit être responsive (mobile-first).
5. Aucune dépendance externe à des CDN sauf si explicitement demandé.
6. Chaque section de code est précédée d'un commentaire explicatif.
7. Si une fonctionnalité est ambiguë, propose deux options et attends.

CONTEXTE DU PROJET

Application : [NOM DE L'APPLICATION]
Objectif : [CE QUE L'UTILISATEUR DOIT POUVOIR FAIRE]
Utilisateurs : [PROFIL]
Palette : [COULEURS HEX]
Contraintes : [LISTE DES CONTRAINTES]

FORMAT DE RÉPONSE

Fournis toujours : 1) Une explication de l'architecture en 3 lignes max.
2) Le code complet entre balises ``html. 3) Un résumé des fonctionnalités implémentées et des éventuelles limitations.

Décomposer une app en modules fonctionnels

Avant de prompter, dessinez (même sur papier) les modules de votre application. Voici comment décomposer une application de gestion de tâches :

📁 MODULES D'UNE APP GESTION DE TÂCHES

MODULE 1 — STRUCTURE Layout HTML global : header, zone de saisie, liste des tâches, footer avec compteur



MODULE 2 — STYLE CSS : palette #6C63FF, cartes de tâches, animations au survol, responsive mobile



MODULE 3 — LOGIQUE CRUD JS : ajouter, lire, modifier le statut (fait/non fait), supprimer une tâche



MODULE 4 — PERSISTANCE localStorage : sauvegarder l'état, recharger au démarrage, effacer les terminées



MODULE 5 — FILTRES & TRI Filtrer par statut (tout / actives / terminées), trier par date ou priorité

Exemple concret : CDC complet

📄 CDC — APP "TASKFLOW"

Nom : TaskFlow
Objectif : Gérer une liste de tâches personnelles avec priorités
Stack : HTML/CSS/JS vanilla, un seul fichier, pas de backend
Données : localStorage — objet JSON avec id, titre, statut, priorité, date
Utilisateurs : 1 seul utilisateur, desktop + mobile
Design : Minimaliste. #6C63FF (accent), #0D0D0D (fond dark), blanc (texte)

Fonctionnalités MVP :

- Ajouter une tâche (titre + priorité haute/moyenne/basse)
- Cocher une tâche comme terminée
- Supprimer une tâche
- Filtrer par statut et par priorité
- Compteur "X tâches restantes"
- Persistence via localStorage

Hors scope : Pas de multi-utilisateurs, pas de synchronisation cloud

Contraintes : Tout dans index.html. Aucune lib externe. Responsive.

CHAPITRE 03

Le Workflow de Prototypage

Un bon workflow de prototypage est la différence entre une conversation productive avec l'IA et une spirale de corrections interminables. Ce chapitre décrit la boucle complète, de la génération initiale à la validation d'un prototype stable.

Le pipeline Idée → Prototype validé

PIPELINE DE PROTOTYPAGE

ÉTAPE 1 — IDÉE & CDC Rédiger le CDC selon la méthode SPEC. Définir le MVP minimal. Préparer le prompt système.



ÉTAPE 2 — PROMPT INITIAL Envoyer le prompt système + demande du premier module. Récupérer le code HTML/CSS/JS.



ÉTAPE 3 — PRÉVISUALISATION Coller le code dans CodePen, StackBlitz ou un fichier local. Observer le rendu visuel.



ÉTAPE 4 — TEST FONCTIONNEL Tester chaque fonctionnalité manuellement. Identifier les bugs et comportements inattendus.



ÉTAPE 5 — DÉBOGAGE PAR IA Soumettre les erreurs à l'IA avec contexte + message d'erreur exact. Récupérer la correction.



ÉTAPE 6 — ITÉRATION Ajouter le prochain module. Répéter les étapes 2 à 5 jusqu'à validation complète.

Anatomie d'un bloc de code IA

Lorsque l'IA vous répond avec du code, voici comment lire une réponse typique sans être développeur :

🧩 STRUCTURE TYPE D'UNE RÉPONSE DE CODE HTML

```
<!-- SECTION 1 : Structure HTML (squelette de la page) -->
<!DOCTYPE html>
<html lang="fr">
<head>
  <!-- Métadonnées : titre, encodage, responsive -->
  <meta charset="UTF-8">
  <title>TaskFlow</title>
  <style>
    /* SECTION 2 : Styles CSS (apparence visuelle) */
    body { background: #000000; color: white; }
    .task-card { border-radius: 8px; padding: 16px; }
  </style>
</head>
<body>
  <!-- SECTION 3 : Contenu HTML (interface visible) -->
  <div id="app">...</div>

  <script>
    // SECTION 4 : Logique JavaScript (comportement dynamique)
    function addTask(title) { /* ... */ }
  </script>
</body>
</html>
```

💡 COMMENT LIRE UN BLOC DE CODE SANS ÊTRE DÉVELOPPEUR

Vous n'avez pas besoin de comprendre chaque ligne. Concentrez-vous sur :

- **Les commentaires** (lignes qui commencent par // ou entre /* */ ou <!-- -->) : l'IA doit les écrire — s'il n'y en a pas, demandez-lui d'en ajouter
- **Les noms de fonctions** (addTask, deleteItem, etc.) : ils vous indiquent ce que le code fait
- **Les valeurs entre guillemets** : couleurs, textes, URLs — vous pouvez les modifier directement
- **La structure générale** : repérez les 4 sections (HTML, CSS, Contenu, JS) et leur rôle

Outils de prévisualisation

OUTIL	USAGE IDÉAL	AVANTAGES	LIMITE
CodePen	Prototypes rapides, snippets CSS/JS	Interface instantanée, partage facile	Séparation HTML/CSS/JS forcée
StackBlitz	Apps complètes, projets multi-fichiers	Environnement complet, terminal intégré	Connexion requise
JSFiddle	Tests rapides, débogage JS	Simple, historique des versions	Interface vieillissante
Fichier local	Apps autonomes en un fichier HTML	Pas de dépendance, ouvre dans tout navigateur	Pas de partage direct
VS Code + Live Server	Développement sérieux, projets longs	Rechargement automatique, débogage avancé	Installation requise

Notre recommandation pour débutants : Commencez avec un simple fichier `index.html` ouvert dans Chrome. Copiez le code de l'IA, sauvegardez, rafraîchissez. C'est suffisant pour 90% des cas d'usage.

La boucle de débogage assisté par IA

Quand votre application ne fonctionne pas, suivez ce protocole précis pour soumettre l'erreur à l'IA :

- 1 Ouvrez la console du navigateur**
Appuyez sur F12 (ou Cmd+Option+J sur Mac). Allez dans l'onglet "Console". Lisez les messages en rouge.
- 2 Copiez le message d'erreur exact**
Ne reformulez pas l'erreur. Copiez-la mot pour mot, avec le nom du fichier et le numéro de ligne si présent.
- 3 Décrivez le comportement attendu vs observé**
"Je veux que X se passe quand je clique sur Y, mais Z se passe à la place." Soyez précis.
- 4 Fournissez le code complet actuel**
Ne donnez pas juste l'extrait supposément problématique. L'IA a besoin du fichier entier pour comprendre le contexte.
- 5 Demandez une explication + correction**
"Explique pourquoi cette erreur se produit, puis donne-moi le code corrigé complet."

🐛 TEMPLATE DE PROMPT DE DÉBOGAGE

```
## Contexte
J'ai une application [DESCRIPTION]. Voici le code complet :
[COLLER LE CODE ICI]

## Problème observé
Quand je [ACTION], j'obtiens [COMPORTEMENT OBSERVÉ].
J'attends [COMPORTEMENT ATTENDU].

## Message d'erreur console
[COLLER L'ERREUR EXACTE]

## Demande
1. Explique la cause de cette erreur en 2-3 phrases simples.
2. Fournis le code corrigé complet (pas juste l'extrait).
```

Checklist de validation d'un prototype

- ✓ Toutes les fonctionnalités du MVP sont opérationnelles
- ✓ L'application s'affiche correctement sur mobile (largeur 375px) et desktop
- ✓ Les données persistent après rechargement de la page (localStorage)
- ✓ Aucune erreur rouge dans la console du navigateur
- ✓ Les formulaires valident les entrées (champs vides, formats incorrects)
- ✓ L'interface est compréhensible par un utilisateur non technique
- ✓ Le code est sauvegardé dans un fichier versionné (Notion, Google Drive, Git)
- ✓ Une personne externe a testé l'application sans vos explications

Itérer sans casser : versionner ses prompts

Avant chaque modification substantielle, sauvegardez la version actuelle de votre code. Voici un système simple sans Git :

💡 VERSIONNEMENT SIMPLE SANS GIT

Créez un dossier avec cette structure :

- **v1_structure-de-base.html** — après le module HTML/CSS
- **v2_logique-crud.html** — après l'ajout de la logique JS
- **v3_persistence.html** — après localStorage
- **v4_filtres.html** — après les fonctionnalités avancées
- **prompts/** — dossier contenant tous vos prompts en .txt

En cas de régression, vous pouvez toujours revenir à la version précédente.

CHAPITRE 04

L'Hébergement & la Mise en Ligne

Votre prototype fonctionne localement. Il est temps de lui donner une URL publique et de le rendre accessible à vos utilisateurs. Ce chapitre couvre les méthodes les plus rapides et les plus fiables pour déployer une application web générée par IA.

Les trois chemins vers le web

Il existe trois grandes approches pour héberger une application web, classées par complexité croissante :

- **Hébergement statique** : votre application est un ou plusieurs fichiers (HTML, CSS, JS, images) servis directement. Pas de serveur, pas de base de données côté serveur. C'est idéal pour tout ce que nous avons construit ici. C'est gratuit, rapide et fiable chez Netlify, Vercel ou GitHub Pages.
- **Hébergement mutualisé** : un serveur partagé avec PHP et une base de données. Plus complexe, nécessaire seulement si votre app a besoin d'un backend (login, données partagées entre utilisateurs).
- **Cloud (AWS, GCP, Render)** : pour des applications à fort trafic, des APIs ou des apps avec intelligence artificielle côté serveur. Réservé aux besoins avancés.

Pour 95% des projets générés par IA avec ce guide, l'hébergement statique suffira parfaitement.

Netlify Drop : votre app en ligne en 30 secondes



VOTRE APP EN LIGNE EN MOINS DE 5 MINUTES

Netlify Drop est la solution la plus simple qui existe : vous glissez-déposez un dossier sur une page web et obtenez une URL HTTPS publique en quelques secondes. Aucun compte requis pour le premier déploiement.

1

Préparez votre dossier

Créez un dossier sur votre bureau. Renommez votre fichier de code en "index.html" et placez-le dedans. Ajoutez les éventuelles images dans un sous-dossier "images/".

2

Allez sur app.netlify.com/drop

Ouvrez votre navigateur et rendez-vous sur cette URL. Vous verrez une zone de dépôt verte.

3 Glissez-déposez le dossier

Faites glisser votre dossier entier (pas juste le fichier index.html) dans la zone verte de Netlify.

4 Attendez le déploiement

Netlify traite votre dossier en 5 à 15 secondes et génère automatiquement une URL du type "random-name-12345.netlify.app".

5 Partagez votre URL

Copiez l'URL affichée. Votre application est accessible dans le monde entier, sur HTTPS, gratuitement.

6 (Optionnel) Créez un compte pour conserver l'URL

Sans compte, le site expire après 24h. Créez un compte gratuit Netlify pour le conserver indéfiniment et pouvoir redéployer.

Vercel & GitHub Pages : pour aller plus loin

Vercel est la plateforme de référence pour les applications React/Next.js, mais elle excelle aussi pour les sites statiques. Son avantage principal : une intégration native avec GitHub qui permet le déploiement automatique à chaque modification de code. Idéal dès que votre projet devient un peu plus sérieux.

GitHub Pages est l'option la plus technique des trois mais aussi la plus pérenne. Elle nécessite de créer un dépôt GitHub (gratuit), d'y pousser votre code, puis d'activer Pages dans les paramètres. L'URL est du type *username.github.io/nom-du-projet*. Avantage : votre code est versionné et sauvegardé automatiquement.

Nommer son domaine et configurer un DNS

Une URL en ".netlify.app" est fonctionnelle mais peu professionnelle. Pour un domaine personnalisé :

- Achetez un domaine sur **Namecheap**, **OVH** ou **Google Domains** (environ 10–15€/an)
- Dans le panneau DNS de votre registrar, ajoutez un enregistrement CNAME pointant vers votre URL Netlify
- Dans Netlify, allez dans Site settings → Domain management → Add custom domain
- Netlify génère automatiquement un certificat HTTPS gratuit via Let's Encrypt (5 à 10 minutes)

Sécurité basique : ce qu'il ne faut jamais faire

⚠ NE JAMAIS EXPOSER SES CLÉS API DANS LE CODE FRONTEND

Si votre application utilise une API (OpenAI, Stripe, SendGrid...), sa clé secrète ne doit JAMAIS apparaître dans votre fichier HTML/JS. Ce fichier est lisible par n'importe qui via "Afficher le code source". Utilisez toujours :

- Les **variables d'environnement** de Netlify ou Vercel pour stocker les clés côté serveur
- Des **Netlify Functions** (serverless) pour faire les appels API depuis le serveur, pas depuis le navigateur
- Un **proxy backend** si votre app nécessite plusieurs appels API sécurisés

Tableau comparatif des plateformes

PLATEFORME	FACILITÉ	GRATUIT	DOMAINE PERSO	CI/CD	IDÉAL POUR
Netlify Drop	★★★★★	Oui	Oui (payant)	Manuel	Démarrage immédiat
Netlify (compte)	★★★★★	Oui	Oui	Via GitHub	Projets durables
Vercel	★★★★★	Oui	Oui	Automatique	React/Next.js
GitHub Pages	★★★★	Oui	Oui	Via Git	Devs + portfolio
Render	★★★	Partiel	Oui	Automatique	Apps avec backend

Notre recommandation : commencez avec Netlify Drop pour le premier déploiement, créez un compte gratuit Netlify dès que vous voulez garder votre site en ligne, et passez à Vercel + GitHub quand vous êtes prêt pour un workflow professionnel.

CHAPITRE 05

L'Extension Mobile

Une fois votre application web déployée, la prochaine étape naturelle est de la rendre disponible sur smartphone — idéalement comme une vraie app installable. Ce chapitre explore les deux grandes voies : la PWA (Progressive Web App) et la conversion en APK Android.

Du web au mobile : PWA vs APK natif

Il existe une distinction fondamentale entre deux approches :

- **PWA (Progressive Web App)** : votre application web devient "installable" sur l'écran d'accueil d'un smartphone. Elle s'ouvre en plein écran, sans barre d'adresse, fonctionne hors ligne, peut recevoir des notifications push. Elle reste techniquement une page web — mais l'utilisateur ne le perçoit pas. C'est la solution la plus simple et la plus rapide.
- **APK natif (Android Package)** : un vrai fichier d'installation Android (.apk) que l'utilisateur télécharge et installe comme n'importe quelle application du Play Store. Plus complexe à produire, mais distributable via le Google Play Store officiel.

💡 QUELLE APPROCHE CHOISIR ?

Commencez par la PWA. Elle couvre 80% des besoins mobiles avec 10% de l'effort d'un APK. Passez à l'APK uniquement si vous avez besoin d'une distribution via le Play Store officiel, d'accès aux capteurs natifs (Bluetooth, NFC) ou d'une app pour iOS via React Native.

PWA : les deux fichiers essentiels

Transformer une web app en PWA nécessite d'ajouter deux fichiers à votre projet et quelques lignes dans votre HTML :

1. Le manifest.json

 MANIFEST.JSON — MÉTADONNÉES DE L'APP MOBILE

```
{
  "name": "TaskFlow",
  "short_name": "TaskFlow",
  "description": "Gestionnaire de tâches intelligent",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#000000",
  "theme_color": "#6C63FF",
  "orientation": "portrait",
  "icons": [
    { "src": "/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icon-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

2. Le service-worker.js

 SERVICE-WORKER.JS — CACHE ET MODE HORS-LIGNE

```
const CACHE = 'taskflow-v1';
const FILES = ['/', '/index.html', '/icon-192.png'];

// Installation : mise en cache des fichiers essentiels
self.addEventListener('install', e => {
  e.waitUntil(caches.open(CACHE).then(c => c.addAll(FILES)));
});

// Interception des requêtes : serve depuis le cache
self.addEventListener('fetch', e => {
  e.respondWith(
    caches.match(e.request).then(r => r || fetch(e.request))
  );
});
```

3. Lier le manifest dans votre HTML

 AJOUT DANS INDEX.HTML — <HEAD>

```
←!— Lien vers le manifest —>
<link rel="manifest" href="/manifest.json">
<meta name="theme-color" content="#6C63FF">
<meta name="apple-mobile-web-app-capable" content="yes">

←!— Enregistrement du service worker —>
<script>
  if ('serviceWorker' in navigator) {
    navigator.serviceWorker.register('/service-worker.js');
  }
</script>
```

APK en 10 étapes avec PWABuilder

APK EN 10 ÉTAPES SANS LIGNE DE CODE

PWABuilder (outil officiel Microsoft) permet de convertir n'importe quelle PWA hébergée en APK Android prêt pour le Play Store :

1 Hébergez votre PWA sur Netlify ou Vercel

PWABuilder a besoin d'une URL HTTPS publique. Suivez le chapitre 4 pour déployer d'abord.

2 Allez sur pwabuilder.com

Entrez l'URL de votre app dans le champ de saisie et cliquez sur "Start".

3 Vérifiez le score PWA

PWABuilder analyse votre app et affiche un score. Visez un score > 70. Corrigez les warnings si nécessaire.

4 Cliquez sur "Package for stores"

Sur la page de résultat, cliquez sur le grand bouton "Package for stores".

5 Sélectionnez Android

Choisissez "Android" dans les options de plateforme.

6 Renseignez les métadonnées

Nom de l'app, identifiant (com.votreapp.nom), version, couleurs. Ces infos apparaîtront sur le Play Store.

7 Générez la clé de signature (keystore)

PWABuilder crée automatiquement un keystore. Sauvegardez-le précieusement — vous en aurez besoin pour les mises à jour futures.

8 Téléchargez le package Android

Vous recevez un ZIP contenant le fichier APK et le bundle AAB (requis par le Play Store).

9 Testez l'APK sur votre téléphone

Activez "Sources inconnues" dans les paramètres Android, puis transférez et installez l'APK pour valider.

10 Soumettez sur Google Play

Créez un compte développeur Google Play (25\$ une fois), uploadez le bundle AAB et remplissez la fiche store.

Prérequis Google Play Store

- **Compte développeur** : 25\$ une seule fois (play.google.com/console)
- **Icônes** : au minimum 512×512px en PNG, fond uni
- **Screenshots** : 2 à 8 captures d'écran de l'app sur smartphone
- **Description** : courte (80 car.) + longue (4000 car.) en français et anglais
- **Politique de confidentialité** : URL obligatoire — générez-en une avec [PrivacyPolicies.com](https://www.privacypolicies.com)
- **Avis de contenu** : questionnaire à remplir (5 minutes)

Expo & React Native : quand passer à l'étape suivante

Si votre application nécessite des fonctionnalités natives avancées (caméra, GPS précis, Bluetooth, notifications locales avancées, app iOS native), il faudra passer à **Expo** ou **React Native**. Ces outils permettent de créer de vraies apps natives en JavaScript, tout en restant accessibles avec l'aide de l'IA.

L'IA peut générer du code Expo complet. Le workflow reste le même : CDC → prompt → code → test → déploiement. La différence est que vous devrez installer Node.js et utiliser Expo Go pour tester sur votre téléphone.

iOS et l'App Store : la voie alternative

⚠️ IOS : LA VOIE PWA EST RECOMMANDÉE

Soumettre sur l'App Store d'Apple nécessite un Mac, un compte Apple Developer à 99\$/an, et un processus de validation plus strict. Pour les non-développeurs, nous recommandons fortement la PWA sur iOS. Sur iPhone, l'utilisateur peut "Ajouter à l'écran d'accueil" depuis Safari — l'expérience est quasi-identique à une app native. Testflight (bêta Apple) permet de distribuer à des testeurs sans passer par l'App Store.

CHAPITRE 06

Cheat Sheet Finale

Condensé de tout le guide en un seul chapitre de référence. À garder ouvert pendant vos sessions de codage assisté par IA.

Les 10 règles d'or du codage assisté par IA

01 Toujours fournir un contexte complet

L'IA génère du code proportionnel à la qualité de votre brief. Sans contexte, elle devine — et se trompe. Utilisez toujours la méthode SPEC avant tout prompt de code.

02 Un module à la fois

Ne demandez jamais de construire une app entière en un seul prompt si elle a plus de 5 fonctionnalités. Décomposez, construisez module par module, validez chaque étape avant d'avancer.

03 Nommez les comportements, pas les technologies

"Quand je clique sur le bouton, la tâche s'ajoute à la liste et le champ se vide" est meilleur que "utilise un event listener onClick pour push dans le tableau". Décrivez ce qui doit se passer, l'IA choisit comment.

04 Toujours demander le code complet

Après chaque correction ou ajout, demandez explicitement "fournis-moi le fichier HTML complet mis à jour". Les extraits partiels créent des erreurs d'intégration difficiles à déboguer.

05 Tester avant d'itérer

Ne demandez jamais la prochaine fonctionnalité avant d'avoir validé la précédente dans le navigateur. Une base instable rend les iterations exponentiellement plus difficiles.

06 Sauvegarder chaque version qui fonctionne

Avant toute modification significative, sauvegardez le fichier actuel avec un numéro de version. En cas de régression, vous pouvez toujours revenir en arrière en 10 secondes.

07

Utiliser la console navigateur comme premier outil de diagnostic

F12 → Console est votre meilleur allié. Toute erreur y apparaît en rouge avec la ligne exacte. Copiez ce message verbatim dans votre prompt de débogage — ne reformulez jamais.

08

Ne jamais exposer les clés API dans le code frontend

Tout code HTML/JS est lisible publiquement. Utilisez les variables d'environnement de Netlify/Vercel ou des fonctions serverless pour les appels API sécurisés.

09

Demander des explications, pas juste le code

Ajoutez toujours "explique les choix architecturaux en 3 lignes" à votre prompt. Comprendre la logique générale vous rend autonome pour les prochaines itérations.

10

Relancer la conversation si elle dérive

Après 15-20 échanges, le contexte de l'IA se dégrade. Si les réponses deviennent incohérentes, ouvrez une nouvelle conversation en re-injectant le prompt système + le code actuel.

Les 7 erreurs fatales à éviter

x1

Prompter sans CDC

Sans structure préalable, l'IA génère une app qui ne correspond pas à vos besoins. Résultat : des heures de corrections inutiles pour un résultat qui aurait pris 20 minutes avec un bon CDC.

x2

Copier du code sans comprendre sa structure

Copier un code sans en identifier les 4 sections (HTML, CSS, Contenu, JS) rend tout débogage impossible. Prenez 2 minutes pour lire les commentaires avant de tester.

x3

Ignorer les erreurs de console

Une erreur en console = une fonctionnalité cassée qui va contaminer les suivantes. Ne jamais "avancer malgré" une erreur rouge. Réglez-la immédiatement avant d'ajouter du code.

x4

Mélanger plusieurs fichiers sans organisation

Commencer avec un fichier unique index.html, puis ajouter des fichiers CSS et JS séparés sans structure claire crée une confusion qui rend le code impossible à maintenir par l'IA.

x5

Déployer avant de tester localement

Netlify n'est pas un environnement de test. Validez toujours votre application localement (double-clic sur index.html) avant de déployer. Les erreurs de prod sont plus difficiles à déboguer.

x6

Reformuler les messages d'erreur

"J'ai une erreur sur la fonction d'ajout" donne des corrections à l'aveugle. "Uncaught TypeError: Cannot read properties of undefined (reading 'push') at addTask (index.html:47)" donne une correction précise et immédiate.

x7

Ne pas versionner son travail

Sans sauvegarde des versions, une seule mauvaise correction peut vous faire perdre des heures de travail. Un simple dossier avec des fichiers numérotés v1, v2, v3 suffit à éviter cette catastrophe.

Le template de prompt ultime réutilisable

 PROMPT MAÎTRE — À COPIER ET ADAPTER POUR CHAQUE PROJET**## RÔLE**

Tu es un développeur full-stack senior. Tu codes proprement en HTML/CSS/JS vanilla, tout dans un seul fichier autonome.
Tu commentes chaque section importante. Tu choisis toujours la solution la plus simple et la plus lisible.

PROJET

Nom : [NOM]
Description : [CE QUE L'APP FAIT EN 1 PHRASE]
Utilisateur : [QUI L'UTILISE, SUR QUEL APPAREIL]

FONCTIONNALITÉS (MVP uniquement)

1. [FONCTIONNALITÉ 1]
2. [FONCTIONNALITÉ 2]
3. [FONCTIONNALITÉ 3]

DESIGN

Style : [MINIMALISTE / COLORÉ / CORPORATE]
Palette : [COULEUR PRINCIPALE], [COULEUR SECONDAIRE], [FOND]
Police : [INTER / ROBOTO / SYSTEM-UI]
Responsive : Oui, mobile-first

DONNÉES

Stockage : [localStorage / aucun / API externe]
Structure : [{ id, titre, statut, date }]

CONTRAINTES ABSOLUES

- Un seul fichier HTML, aucune dépendance externe
- Pas de framework JavaScript
- Commentaires sur chaque fonction JS importante
- Validation des formulaires (champs vides bloqués)

FORMAT DE RÉPONSE

1. Architecture en 3 lignes
2. Code complet entre ``html
3. Liste des fonctionnalités implémentées

Ressources et outils recommandés

CATÉGORIE	OUTIL	USAGE	PRIX
IA Codage	Claude (Anthropic)	Génération et débogage de code	Gratuit / Pro 20\$/mois
IA Codage	GPT-4o (OpenAI)	Alternative, très capable	Gratuit / Plus 20\$/mois
IA Codage	GitHub Copilot	Autocomplétion dans VS Code	10\$/mois
Prévisualisation	CodePen	Test rapide HTML/CSS/JS	Gratuit
Prévisualisation	StackBlitz	Projets complets en ligne	Gratuit
Hébergement	Netlify	Déploiement statique	Gratuit (100GB)
Hébergement	Vercel	Déploiement React/Next.js	Gratuit
Domaines	Namecheap	Achat de nom de domaine	~10€/an
Mobile	PWABuilder	Conversion PWA → APK	Gratuit
Mobile	Expo	Apps React Native	Gratuit
Icônes	Favicon.io	Générer icônes PWA	Gratuit
Éditeur code	VS Code	Éditeur local + Live Server	Gratuit

Lexique express — 10 termes clés

LLM (Large Language Model)

Modèle d'intelligence artificielle entraîné sur des milliards de textes, capable de générer, comprendre et transformer du texte et du code. Claude, GPT-4, Gemini sont des LLM.

Prompt

L'instruction ou la question que vous envoyez à un LLM. La qualité du prompt détermine directement la qualité de la réponse. C'est le coeur du "prompt engineering".

HTML / CSS / JS

Les trois langages de base du web. HTML = structure (les boîtes), CSS = style (les couleurs et formes), JavaScript = comportement (ce qui bouge et réagit).

localStorage

Mécanisme du navigateur permettant de sauvegarder des données directement sur l'appareil de l'utilisateur, sans serveur. Les données persistent après rechargement de la page.

Deploy / Déploiement

Action de rendre une application accessible sur internet via une URL publique. Netlify, Vercel et GitHub Pages sont des plateformes de déploiement.

PWA (Progressive Web App)

Application web conçue pour s'installer sur l'écran d'accueil d'un smartphone et fonctionner comme une app native, y compris hors ligne.

APK

Android Package Kit — le format de fichier d'installation des applications Android. Un fichier .apk s'installe sur un téléphone Android comme un programme sur un ordinateur.

MVP (Minimum Viable Product)

La version minimale fonctionnelle d'un produit. Dans le contexte du codage IA, c'est la liste restreinte de fonctionnalités à implémenter en priorité pour valider le concept.

Console navigateur

Panneau de développement accessible via F12 (ou Cmd+Option+J sur Mac). Affiche les erreurs JavaScript en rouge. Outil de diagnostic indispensable pour le débogage.

CRUD

Create, Read, Update, Delete — les quatre opérations fondamentales sur des données. Toute application de gestion (tâches, contacts, inventaire) est essentiellement un CRUD.

POUR ALLER PLUS LOIN AVEC AI MASTERY

Ce guide couvre les fondations du développement assisté par IA. La masterclass AI Mastery approfondit chaque étape avec des ateliers pratiques, des projets guidés de A à Z, et une communauté de praticiens actifs. Chaque module est construit sur le même principe : comprendre le pourquoi, maîtriser le comment, pratiquer immédiatement.

aimastery.io — Rejoignez la prochaine cohorte.